CV Intern Task

Egg segmentation, evaluation and video tracking: report for Tasks 1 to 4

Author: Anant Singla

Repository: https://github.com/AnantSingla412/Nosh-CV

Task 4 output video (Google Drive):
https://drive.google.com/file/d/1J1LyZ5amhyR-DFS8jQ2IINm2QukMPneT/view?usp=sharing

Summary

Task	What was done	Key result
1. Pretrained segmentation	OWL-ViT finds egg boxes; SAM ViT-B segments each box.	110 images, 155 egg masks; used as the reference for the other tasks.
2. Classical segmentation	Group-safe 79/20 dev/eval split; 11 documented trials from Otsu to an Auto-Context Random Forest with hole filling and a component shape filter.	Eval IoU `0.809`, Dice `0.853` (20 images); dev out-of-fold IoU `0.776` (Otsu baseline `0.335`).
3. Evaluation and analysis	Pixel, boundary and egg-level metrics with bootstrap intervals; failure categories, error drivers and failure galleries.	Errors concentrate where the egg has low colour contrast with its surroundings (mean IoU `0.64` against `0.89` in the highest-contrast third); touching eggs merge.
4. Video tracking	Candidate filtering and the Task 2 pipeline on zoomed crops, then a Kalman tracker with occlusion handling and re-identification.	Two egg tracks across the whole egg segment (`32 s`); masks agree with Task 1 at macro IoU `0.913` on 39 sampled frames.

Contents

- Task 1: Segmentation using a Pretrained Model
- Task 2: Segmentation using Image Processing (experiment log)
- Task 3: Evaluation and Analysis
- Task 4: Video Tracking
- Setup and Reproduction

Task 1: Segmentation using a Pretrained Model

Eggs in all 110 dataset images are segmented with a pretrained detector (OWL-ViT) to find egg boxes and the Segment Anything Model (SAM ViT-B) to turn each box into a mask. These masks are the reference ("ground truth") for Tasks 2 and 3 and the comparison point in Task 4. Script: `scripts/task1_segment.py`; results: `outputs/task1_final/`.

Pipeline

1. OWL-ViT (`google/owlvit-base-patch32`) detects eggs with the text prompt `egg`.
2. Detections below the score threshold (`0.02`) are rejected, boxes covering more than `60%` of the image are removed, and duplicate boxes are removed with non-maximum suppression (IoU `0.45`).
3. If no box remains, the script retries with the prompt `boiled egg` (score threshold `0.008`) and keeps the single best detection.
4. SAM receives the image and each box and proposes mask candidates; the candidate with the highest SAM-predicted quality is kept for each box. A combined mask and an overlay are saved per image.

Threshold selection

Thresholds were compared on five test images. A detector score of `0.08` missed eggs. `0.02` found more, but a loose box-area limit admitted a bowl-sized false detection, which led to the `60%` area limit. These settings improve coverage but do not remove all false detections.

Results

Source split	Images	Egg masks
Train (75 egg images and 2 unlabeled images)	77	110
Validation	22	33
Test	11	12
Total	**110**	**155**

Every image received at least one mask. Egg counts per image: `1` egg in `85` images, `2` in `11`, `3` in `10`, `4` in `2` and `5` in `2`.



The four overlays above are evenly spaced through the development and evaluation images in file order, not hand-picked. Each shows the detection box with its score and the SAM mask.

Limitations

- **False detections remain.** A manual review of the masks found ten images where Task 1 produced a false positive; these ten images and one duplicate unlabeled image were excluded from the Task 2 split (see the Task 2 log), leaving 99 images for Tasks 2 and 3.
- **Task 1 is not perfect ground truth.** Because Tasks 2 to 4 use these masks as the reference, any Task 1 error carries into every later score.
- **No ground truth was annotated by hand.** The masks come from the models; no human segmentation exists for comparison.

Run

```
python scripts/task1_segment.py
```

The OWL-ViT model downloads on first use, and the SAM ViT-B checkpoint (`sam_vit_b_01ec64.pth`) must be placed in `.venv/model_cache/`. Paths can be overridden with `--image-dir`, `--output-dir` and `--sam-checkpoint`.

Task 2: Segmentation using Image Processing (experiment log)

Task 1 combined masks are used as reference masks for the pixel metrics.

Summary: from baseline to final method

Trial	Change	Dev IoU	Outcome
1	Global Otsu threshold on Lab lightness	0.335	Baseline
2	Gaussian adaptive threshold	0.077	Discarded
3	GrabCut initialized from Otsu	0.330	Kept as a trial, not selected
4	Random Forest pixel classifier	0.332*	Kept as the starting candidate
5	Group-safe out-of-fold threshold (0.65)	0.406	Cutoff kept
6	Context-aware features	0.469	Kept
7	Color-guided spatial refinement	0.474	Kept; small gain
8	Multiscale filter-bank features (variant C)	0.555	Kept
9	Mask cleanup: hole filling	0.606	Hole filling kept; other variants discarded
10	Scale fix + Auto-Context	0.718	Kept
11	Component shape filter + keep-best	0.776	Final method
Eval	Final method, 20 held-out images	0.809	Reported once

Dev IoU is the macro IoU over the 79 development images from out-of-fold predictions for Trials 5-11 and a direct run for Trials 1-3. *Trial 4 was measured on a 16-image validation subset only, so it is not comparable with the other rows.

Data split and duplicate review

- Ten user-identified images and one duplicate unlabeled image were excluded from the split. The source dataset remains unchanged.
- The first duplicate check used a perceptual hash. It grouped `not_damaged_27` / `not_damaged_78`, but missed the differently framed `not_damaged_135` / `not_damaged_145` pair. This led to the simpler full-image CLIP comparison.
- The split script extracts one normalized CLIP ViT-B/32 embedding from each full image and compares every pair with cosine similarity. Pairs at or above the selected cutoff are joined into similarity groups; transitive links form a single group. All retained members of a group go to development, and evaluation is selected only from singleton images.
- **Increment: label candidate pairs and compare cutoffs.** We visually labeled 28 high-similarity pairs (scores `>=0.950`) as same-scene or different-scene; the labels and brief reasons are in `reports/task2_clip_pair_review.csv`. On this reviewed set, cutoff `0.950` gave precision `0.9286`, recall `1.0000`, and F1 `0.9630` (26 same-scene pairs found, 2 different-scene pairs grouped). At `0.955`, F1 was `0.8511` (20 found, 1 different-scene pair grouped, 6 missed from the reviewed set). We selected `0.950` for the revised split because it had the best F1 on these candidates and retained all reviewed same-scene pairs. This recall is only within the reviewed high-similarity set; pairs below `0.950` were not comprehensively labeled.
- The selected `0.950` threshold groups 28 pairs into similarity groups. It catches the known `not_damaged_27` / `not_damaged_78` pair (`0.987935`) and the retained `not_damaged_124` / `not_damaged_135` scene (`0.978973`); the duplicate unlabeled `145` remains excluded. The review also found different-scene pairs `37/55` (`0.955334`) and `146/33` (`0.953865`), showing that CLIP can over-group. Groups are kept out of evaluation to favor leakage reduction.
- Similarity distribution analysis covered 5,995 pair scores: median `0.7914`, 95th percentile `0.8990`, and 99th percentile `0.9394`. Scores taper through the cutoff region with no clean separation, so we used the labeled candidate review to compare nearby cutoffs.

- Final split: 79 development images, 20 singleton evaluation images, and 11 excluded images. No CLIP similarity group crosses development and evaluation. The manifest records the groups and the ignored `outputs/task2/clip_pair_similarities.csv` records pair scores.
- The histogram and contact sheets are generated by `scripts/task2_similarity_analysis.py` and remain local under `outputs/task2/similarity_analysis/`. CLIP thresholding can over-group similar images and can miss duplicates below the cutoff. The Otsu results below were collected on the earlier development split and were not rerun after this split change.

Trial: three overlapping crops

- **Change:** Along with the full image, extract three 75% crops positioned at the top-left, center, and bottom-right. For each image pair, combine full-image cosine similarity with the maximum crop-to-crop cosine similarity: `0.5 × full + 0.5 × crop maximum`.
- **Evaluation:** Use the same 28 labeled candidate pairs as the full-image method. The best crop score F1 was `0.963` at thresholds `0.900–0.930`, equal to the full-image F1 on this sample. At `0.930`, the crop score flagged 100 of 5,995 pairs and formed 14 groups; full-image CLIP at `0.950` flagged 28 pairs and formed 7 groups.
- **Decision:** The crops did not improve the reviewed-pair result and produced substantially more groups, suggesting more over-grouping. Keep full-image CLIP as the default split method; `--three-crops` remains available to reproduce this trial. The crop-pair scores are saved locally in `outputs/task2/clip_pair_similarities_three_crop_trial.csv`. No Task 2 split or Otsu result was changed for this trial.

Trial 1: Global Otsu threshold on Lab lightness

Reason: Establish a fast, interpretable brightness-only baseline before adding local or shape-based rules.

Method and parameters: Convert each image to CIE Lab, apply binary Otsu thresholding to the L channel, and use the resulting mask as-is. No blur, morphology, contour filtering, or instance splitting is applied. Run on development images only.

Development results: 79 images; macro IoU `0.3346`, Dice `0.4568`, precision `0.3644`, recall `0.8859`. Mean Otsu threshold was `127.4` on the 8-bit L channel.

Examples and shortcomings: The isolated eggs in `not_damaged_148` (IoU `0.963`) and `not_damaged_115` (`0.953`) were segmented closely. In `not_damaged_24` (IoU `0.0004`), the bright wall was labeled as foreground and the egg was mostly missed. In `not_damaged_117` (`0.0216`), much of the egg carton was selected. The global brightness threshold finds many reference pixels but also includes bright background regions.

Decision: Keep as the baseline, not as the final method. Its high recall and low precision show that the next trial should use local image context.

Trial 2: Gaussian adaptive threshold on Lab lightness

Reason: Test whether local brightness context can reduce the broad background detections seen with global Otsu thresholding.

Method and parameters: Apply Gaussian adaptive thresholding to Lab L using a `51 x 51` neighborhood and `C = -5`. No blur or morphology is applied. The threshold therefore varies with local brightness rather than using one image-wide value.

Development results: 79 images; macro IoU `0.0771`, Dice `0.1398`, precision `0.1624`, recall `0.1777`. The existing Otsu baseline scored IoU `0.3346`, Dice `0.4568`, precision `0.3644`, recall `0.8859`. Adaptive thresholding improved per-image IoU on 5 images and reduced it on 74. Otsu results were read from the prior run; Otsu was not rerun after the split change.

Examples and shortcomings: The method often labeled textured surroundings as foreground while missing egg interiors, visible as red false-positive regions and blue false-negative regions in the comparison overlays. It did not solve the bright-wall error in `not_damaged_24` or the carton error in `not_damaged_117`; overall, it lost too much egg coverage.

Decision: Discard this configuration. Local thresholding alone performs substantially worse than Otsu on this dataset, likely because egg surfaces are locally uniform and background texture creates many local intensity changes.

Trial 3: GrabCut initialized from Otsu

Reason: Test whether GrabCut's foreground/background color models can refine Otsu's rough mask and remove background regions while retaining egg pixels.

Method and parameters: Convert Lab-L Otsu output into GrabCut labels: Otsu foreground pixels are probable foreground, a `5 x 5` eroded core is sure foreground, and other pixels are probable background. The image border (1% of the shorter dimension) is marked sure background. Run OpenCV GrabCut for 5 iterations and combine definite and probable foreground labels. No additional packages were needed.

Development results: Processed all 79 current development images. Macro IoU `0.3296`, Dice `0.4473`, precision `0.3433`, recall `0.9259`. The saved Otsu run predates the latest split, so the direct comparison uses only 76 image paths present in both results: Otsu IoU `0.3251`, Dice `0.4476`, precision `0.3512`, recall `0.8876`; GrabCut IoU `0.3290`, Dice `0.4465`, precision `0.3420`, recall `0.9314`. GrabCut improved IoU on 45 matched images and reduced it on 31. Otsu was not rerun.

Examples and shortcomings: GrabCut improved IoU for `not_damaged_33` (`0.693` to `0.864`) and `not_damaged_21` (`0.877` to `0.981`), but reduced it for `not_damaged_95` (`0.307` to `0.226`) and `not_damaged_62` (`0.134` to `0.079`). Overall gains are small and inconsistent: recall rises while precision and Dice fall slightly. The Otsu initialization still limits the color model when it incorrectly selects background or omits egg regions. Processing is also slower than direct thresholding.

Decision: Keep as a documented trial, but do not select it as the final method yet. It yields a small IoU increase on the shared comparison images, without a consistent improvement across metrics or examples.

Trial 4: Random Forest pixel classifier

Reason: The threshold and GrabCut methods rely on per-image brightness or initialization. Train a classical classifier on color and local texture examples from development images so it can learn egg/background appearance across scenes.

Method and parameters: Extract Lab color, `9 x 9` local grayscale standard deviation, and grayscale gradient magnitude per pixel. Use 1,200 foreground and 1,200 background samples per training image after resizing images to at most 320 pixels on the long side. Train an 80-tree Random Forest (`max_depth=18`, `min_samples_leaf=4`, balanced subsample class weights). Remove connected components smaller than `0.01%` of image area (at least 16 pixels). A deterministic group-safe split uses 63 development images for training and 16 for validation; no image similarity group crosses the split. The evaluation split is untouched.

Development results: On the 16 validation images, macro IoU `0.3324`, Dice `0.4561`, precision `0.3853`, recall `0.7859`. On those exact same images, the saved Otsu results were IoU `0.2177`, Dice `0.3269`, precision `0.2286`, recall `0.9265`. The classifier improved IoU and Dice on 14 of 16 images and precision on 13; recall improved on 2. Otsu scores were read from the existing output and were not rerun.

Examples and shortcomings: `not_damaged_95` improved from IoU `0.307` with Otsu to `0.793`; `not_damaged_160` improved from `0.085` to `0.388`. `not_damaged_21` remained strong (`0.877` Otsu, `0.891` classifier). The classifier is more precise but misses more reference pixels overall, and a few validation images remain poor (for example, `not_damaged_116`, IoU `0.092`).

Decision: Keep this as the strongest current candidate. It improves the overlap and precision metrics substantially on the held-out development subset, while the lower recall shows that the `0.5` probability cutoff may be conservative. The split, features, and cutoff are fixed for this recorded trial.

Trial 5: Group-safe out-of-fold threshold selection

Reason: The prior classifier used a fixed `0.5` probability cutoff and only 16 held-out dev images. Evaluate the precision/recall tradeoff across all development images while ensuring each image is predicted by a model that did not train on its similarity group.

Method and parameters: Assign CLIP similarity groups to five deterministic, approximately balanced folds. For each fold, train the same 80-tree Random Forest on the other four folds with 1,200 sampled pixels per class per training image, then save foreground probabilities for the held-out images. Evaluate thresholds from `0.15` through `0.80` in `0.05` increments, applying the same minimum-component cleanup (`0.01%` of image area, minimum 16 pixels). Choose the threshold with the highest macro IoU; ties are resolved by macro Dice, then closeness to `0.5`.

Development results: Out-of-fold predictions cover all 79 dev images. At threshold `0.50`, macro IoU `0.3928`, Dice `0.5303`, precision `0.4689`, recall `0.7641`. The selected `0.65` threshold gave IoU `0.4063`, Dice `0.5463`, precision `0.5438`, recall `0.6673`. The IoU improvement is `0.0135`; precision rises and recall falls. The untouched eval split was not used.

Examples and shortcomings: Increasing the threshold improves precision steadily over the tested range but reduces recall. This does not recover missed egg pixels; it selects a more conservative mask. Since the same out-of-fold predictions select and score the threshold, the best score may be optimistic from threshold selection. It is a development result, not a final eval estimate.

Decision: Keep `0.65` as the best development cutoff to test in the next comparison, but do not change the main classifier default or report it as final until the resulting masks are reviewed and then measured once on the locked eval split.

Trial 6: Context-aware Random Forest

Reason: The first Random Forest used only per-pixel Lab color, local grayscale variation, and gradient. Add broader neighborhood context so predictions can use local appearance patterns at more than one scale.

Method and parameters: Extract per-pixel Lab color, local Lab mean and standard deviation over `9 x 9` and `31 x 31` neighborhoods, and grayscale gradient magnitude. Train the same 80-tree Random Forest with `max_depth=18`, `min_samples_leaf=4`, and balanced subsample class weights. Use 1,200 foreground and 1,200 background samples per training image, resized to a maximum long side of 320 pixels. Use five deterministic folds that keep each CLIP similarity group together. Test thresholds from `0.15` to `0.80` in `0.05` steps; remove components smaller than `0.01%` of the image area (minimum 16 pixels).

Development results: Out-of-fold predictions cover all 79 dev images. The selected threshold was `0.55`, giving macro IoU `0.4693`, Dice `0.6096`, precision `0.6132`, and recall `0.7019`. Compared with the previous pixel classifier's selected cutoff results (IoU `0.4063`, Dice `0.5463`, precision `0.5438`, recall `0.6673`), all four metrics improved. The evaluation split remains untouched.

Decision: Keep this as the strongest development candidate so far. The comparison is promising, but the selected threshold was chosen from the same out-of-fold predictions used to report these metrics, so the score may be optimistic. Evaluate it once on the locked eval split after the method is selected.

Trial 7: Color-guided spatial refinement

Reason: Test whether local spatial consistency can further improve the context-aware Random Forest masks without adding a package.

Method and parameters: For each probability map, apply a joint-bilateral-style local weighted average over a `5 x 5` neighborhood. Spatial weights use sigma `2` pixels; color weights use Euclidean CIE Lab distance with sigma `18` on 8-bit channels. Threshold the refined probabilities at the context model's selected cutoff of `0.55`, then remove connected components below `0.01%` of image area (minimum 16 pixels). The same five group-safe folds and context-aware features are used. This is a dependency-free edge-aware approximation, not a DenseCRF implementation.

Development results: Out-of-fold predictions cover all 79 dev images. Macro IoU `0.4743`, Dice `0.6132`, precision `0.6186`, recall `0.7066`. The context-aware unrefined result at `0.55` was IoU `0.4693`, Dice `0.6096`, precision `0.6132`, recall `0.7019`.

Decision: The refinement gives a small improvement across all four metrics in this dev comparison. Keep it as a candidate; the gain is modest and was measured on the same folds used during development. Review overlays and confirm once on the locked eval split only after selecting the method.

Trial 8: Weka-style multiscale Random Forest features

Reason: The context-only Random Forest improved over simpler methods, but local smoothing added little. Test a broader filter bank to give the classifier multiscale intensity, edge, structure, and texture cues.

Method and parameters: Compare three feature variants with the same 80-tree Random Forest, training pixel samples, 320-pixel maximum training side, and five group-safe folds: (A) Trial 6 context features, (B) multiscale features, and (C) context plus multiscale features. The filter bank uses Lab-L Gaussian smoothing at sigma `1`, `2`, and `4`, gradient magnitude, Laplacian, Hessian eigenvalues, and `9 x 9` local variance at each scale. All variants use threshold `0.55` and the same small-component cleanup. No new dependencies were installed; entropy and Gabor filters were not included in this first comparison.

Development results: On out-of-fold predictions for all 79 dev images, variant A scored macro IoU `0.4679`, Dice `0.6081`, precision `0.6122`, recall `0.7004`; variant B scored IoU `0.5193`, Dice `0.6424`, precision `0.6439`, recall `0.7081`; variant C scored IoU `0.5547`, Dice `0.6804`, precision `0.7080`, recall `0.7240`. The feature variants use the same folds and fixed threshold for a controlled comparison. The eval split remains untouched.

Decision: Keep variant C as the strongest Task 2 candidate so far. Adding the multiscale filter bank to context features improved all four metrics; B also outperformed A. These are development results and still need qualitative review before a final eval run.

Qualitative review of Trial 8 variant C

Reason: Check where the strongest candidate fails before adding another algorithm.

Method: Recomputed per-image metrics from the saved variant C masks and reviewed the 12 lowest-IoU comparison overlays. No model was retrained and the eval split was not used.

Development results: Macro IoU `0.5547`, median `0.5647`; 16 of 79 images score below `0.30` and 15 above `0.80`. A few severe failures lower the mean.

Failure types:

- **Bright, cluttered background selected as egg:** `not_damaged_24`, `117`, `116`, `123`, `20`, `104`, `111`, `119` (IoU `0.02`-`0.26`). White walls, egg cartons, foil and skin are labeled foreground, and masks split into 30-110 fragments where the reference has one to three eggs. White eggs on white or shiny surroundings cannot be separated by local color and texture. In `24` the backlit egg is missed entirely.
- **Correct outline, hollow interior:** `not_damaged_141`, `35`, `37`, `100`, `34`, `101` (precision `0.84`-`0.99`, recall `0.37`-`0.60`). Eggs that fill the frame have featureless centers; the widest context window (`31 x 31`) and filter scale (sigma `4`) do not span them.

Decision: Keep variant C as the candidate. Both failure types suggest mask-level cleanup (hole filling and component filtering) before larger-scale features or Auto-Context.

Trial 9: Mask cleanup on combined-feature probabilities

Reason: The qualitative review found fragmented background detections and hollow egg interiors; test whether mask-level cleanup fixes them without retraining.

Method and parameters: Retrain the combined context + multiscale Random Forest on the same five group-safe folds and threshold at `0.55` (Trial 8 variant C), then compare four variants: (A) baseline cleanup only; (B) fill enclosed holes; (C) morphological closing with an elliptical kernel of about `3%` of the shorter image side, then hole filling; (D) C plus removal of components smaller than `20%` of the largest or with mean probability below `0.6`. No new dependencies; the eval split was not used. The filter values were set once and not tuned.

Development results: Out-of-fold predictions for all 79 dev images.

- A: IoU `0.5546`, Dice `0.6802`, precision `0.7074`, recall `0.7241`.
- B: IoU `0.6058`, Dice `0.7176`, precision `0.7115`, recall `0.7958`.
- C: IoU `0.6111`, Dice `0.7188`, precision `0.6715`, recall `0.8650`.
- D: IoU `0.5884`, Dice `0.6775`, precision `0.6614`, recall `0.7695`.

Hole filling raised IoU on 53 images and lowered it on 8, none by more than `0.02`. Large gains came from the hollow-interior failures (for example `not_damaged_141`, `101`). A first run of this trial had a hole-filling bug (flood fill seeded at a foreground corner pixel turned some masks into the full image); it was fixed by padding the mask, and

the numbers above are from the corrected run.

Shortcomings: Closing raises recall but merges nearby background, lowering precision, and hurts 23 images. The component filter removes genuine low-probability eggs (`not_damaged_34`, `37`, `138`) and is worse than hole filling alone. Remaining errors after hole filling: bright or textured background fragments (`not_damaged_149`, `30`, `27`, `36`, `32`) and large eggs whose interior holes connect to the boundary and are not enclosed (`not_damaged_27`, `36`, `118`, `119`).

Decision: Keep hole filling (B) as the cleanup step; it is the safest gain. Discard D. C has marginally higher IoU and Dice but lower precision and more regressions, so it is not selected.

Trial 10: Scale-consistent Auto-Context

Reason: After hole filling, remaining failures were bright or textured background fragments and large-egg interiors. Checking the pipeline also showed a scale mismatch: all images are `640 x 640`, but every Random Forest was trained on pixels sampled from `320`-pixel images and applied at `640`, so window and filter scales differed by `2x` at test time. Auto-Context adds large-scale spatial context from a first-stage prediction.

Method and parameters: Train and predict at `320` pixels, then upsample probabilities to `640` for scoring against the reference masks. Stage 1 uses the combined context + multiscale features (Trial 8 variant C). Stage 2 is a second Random Forest (same settings) on those features plus the stage-1 probability map, its Gaussian blurs at sigma `4`, `8`, `16`, and a `31 x 31` local maximum. To avoid leakage, stage-1 probabilities for stage-2 training images come from an inner group-safe four-fold out-of-fold prediction inside each outer fold; the five outer folds are the same as before. Threshold `0.55`, small-component cleanup, then optional hole filling. The eval split was not used.

Development results: Out-of-fold predictions for all 79 dev images.

- Stage 1 at `320` (scale fix only): IoU `0.6461`, Dice `0.7509`, precision `0.7262`, recall `0.8540`; with hole filling IoU `0.6677`, Dice `0.7647`.
- Stage 2 (Auto-Context): IoU `0.7114`, Dice `0.7985`, precision `0.7788`, recall `0.8943`; with hole filling IoU `0.7176`, Dice `0.8020`, precision `0.7779`, recall `0.9036`.
- Previous best (Trial 9 hole filling at `640`): IoU `0.6058`, Dice `0.7176`.

With hole filling, the median per-image IoU is `0.81`; 41 of 79 images exceed `0.80` and 10 fall below `0.30` (previously 16).

Examples and shortcomings: The eggs are now found well, including large ones. The remaining low scores (`not_damaged_24`, `117`, `123`, `20`, `116`, `135`, `111`, `32`, `124`) have high recall but precision of `0.10`-`0.38`: large, smooth, bright regions such as white walls, cartons, a steel pot bottom and rim are labeled egg, because no pixel-level feature separates a white egg from a white surface. In `24` the backlit egg is mostly missed and the wall is selected. The threshold was fixed from earlier trials, but stage-2 settings were not tuned; scores remain development results.

Decision: Keep stage 2 with hole filling as the strongest candidate (`+0.11` IoU over Trial 9). The scale fix explains about `0.06` of the gain and Auto-Context the remaining `0.05`.

Trial 11: Component-level shape filtering

Reason: After Auto-Context, the main remaining errors were large, irregular bright regions (walls, cartons, a pot) labeled as egg alongside correctly found eggs. Eggs are compact and convex, so component shape should separate them better than Trial 9's probability and size filter.

Method and parameters: Start from the Trial 10 stage-2 out-of-fold probabilities (threshold `0.55`, cleanup, hole filling) and extract connected components: `508` over the 79 dev images, `120` of them egg-like (at least `50%` of pixels inside the reference mask). Each component is described by area ratio, area relative to the largest component, solidity, extent, overlap with its fitted ellipse, circularity, aspect ratio, border contact, mean and max probability, and centroid. A `200`-tree Random Forest (`max_depth=5`, balanced class weights) predicts egg-like components, trained on the other folds of the same five group-safe folds. Components with keep probability of at least `0.5` are retained. A second variant also keeps the single highest-scoring component when none passes, since every dataset image contains an egg. No new dependencies; the eval split was not used.

Development results: Out-of-fold predictions for all 79 dev images.

- Auto-Context + fill (reference): IoU `0.7176`, Dice `0.8020`, precision `0.7779`, recall `0.9036`.

- Shape filter: IoU `0.7409`, Dice `0.8001`, precision `0.8195`, recall `0.8040`.
- Shape filter + keep best: IoU `0.7764`, Dice `0.8476`, precision `0.8692`, recall `0.8750`.

Keep-best improved IoU on 37 images and lowered it on 5. Median per-image IoU is `0.88`; 48 of 79 images exceed `0.80` and 5 fall below `0.30` (previously 10).

Examples and shortcomings: Many wall and carton false positives were removed. Remaining failures come from three causes:

- **Egg merged with a bright background region:** in `not_damaged_123`, `111`, `104`, `20` and `24` the egg and the wall form one component, which the shape classifier cannot separate (precision `0.14`-`0.35`).
- **Wrong or missing egg:** in `not_damaged_30` a white egg on a white wall is missed and the best-scoring component is a background speck (IoU `0.33` to `0.00`).
- **Multi-egg and partial eggs:** in `not_damaged_103` one of three eggs is discarded (`-0.15`), and `not_damaged_27`, `78` and `11` keep only part of a large egg (precision above `0.97`, recall about `0.4`).

The keep-best fallback assumes at least one egg per image, which may not hold for video frames. The keep probability `0.5` was fixed, not tuned, but shape-model and mask scores are still development results.

Decision: Keep the shape filter with keep-best as the strongest candidate (`+0.06` IoU, `+0.05` Dice over Auto-Context with hole filling). Report the keep-best assumption in the final method description.

Final method and evaluation

Reason: Trial 11 is the strongest dev result and was selected as the final Task 2 method. The evaluation split had been run once earlier by mistake on a candidate pipeline; that run was discarded and no method choice or parameter was changed afterwards. The run reported here is the only evaluation result used.

Method and parameters: Trial 11 unchanged. Both Random Forest stages are trained on all 79 dev images at `320` pixels (stage-2 inputs come from five-fold group-safe out-of-fold stage-1 maps), probabilities are upsampled to `640`, thresholded at `0.55`, cleaned and hole-filled. The component shape classifier is trained on the dev out-of-fold components, keeps components with probability at least `0.5`, and falls back to the single best component when none passes. The `20` singleton eval images were not used for training, thresholds, or model choice.

Evaluation results: 20 images; macro IoU `0.8089`, Dice `0.8531`, precision `0.8457`, recall `0.8927`. Median per-image IoU is `0.93`; 16 of 20 images exceed `0.80` and 3 fall below `0.30`. For comparison, dev out-of-fold IoU was `0.7764` and Dice `0.8476`; the eval set contains 20 singleton images, so the difference is within what a small sample can produce and should not be read as an improvement.

Examples and shortcomings: Isolated eggs on plain backgrounds are segmented closely (IoU `0.94`-`0.99` for `not_damaged_19`, `140`, `156`, `15`, `110`, `127`). The four weakest images repeat the dev failure modes:

- **White egg on a white surface:** in `not_damaged_125` (IoU `0.08`) the egg is mostly missed and a white carton region is kept instead.
- **Egg plus bright background:** in `not_damaged_42` (`0.28`) the egg is found but carton-tray and rod fragments are added; in `not_damaged_112` (`0.70`) white bag material is attached to a correct egg.
- **Very large egg:** in `not_damaged_44` (`0.09`) a close-up egg held in a hand is mostly missed and the hand is selected.

The keep-best fallback assumes at least one egg per frame. Results use Task 1 masks as references, so errors in those masks carry into every score.

Decision: Task 2 is final with the Trial 11 pipeline. Its remaining weakness is separating white eggs from white surfaces, which pixel features and component shape do not resolve; it is reported as a limitation in the Task 3 comparison.

Task 3: Evaluation and Analysis

Task 1 combined egg masks (OWL-ViT + SAM) are treated as ground truth. Predictions are the final Task 2 pipeline (Trial 11: Auto-Context Random Forest, hole filling, component shape filter). All numbers come from `scripts/task3_evaluation.py`; per-image values are in `reports/task3/per_image_metrics.csv`.

Setup

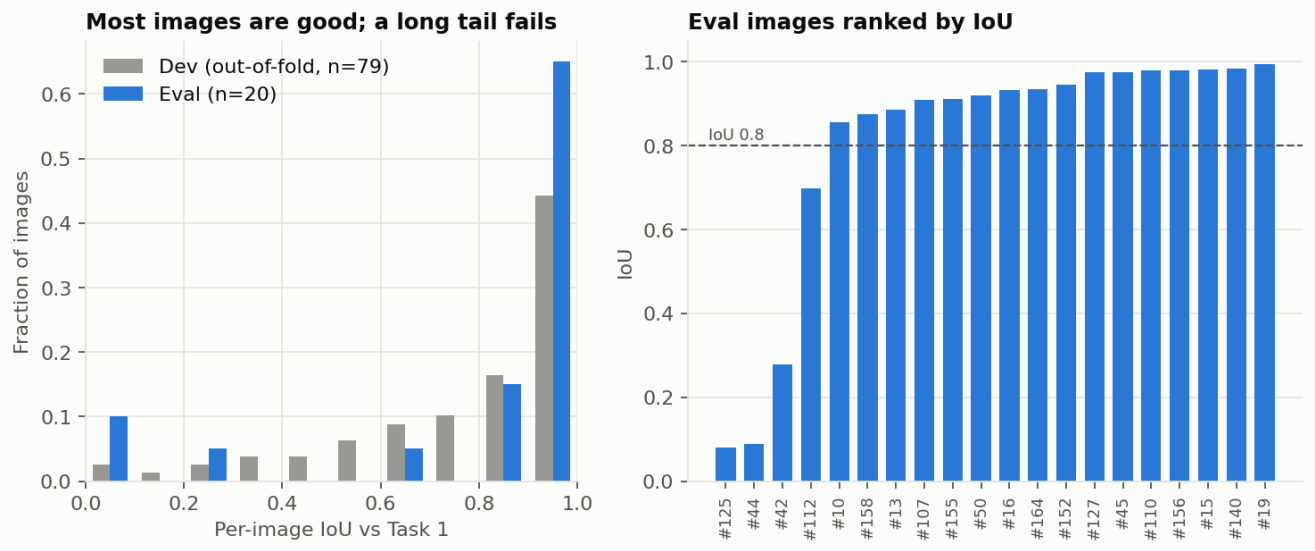
- **Eval split (20 images):** one run of the final pipeline trained on all dev images. This is the unbiased headline number, but 20 images give wide intervals.
- **Dev split (79 images):** out-of-fold predictions, where each image is predicted by a model that never saw its similarity group. Dev was used to choose the method, so it is slightly optimistic, but it provides many more failures to analyse.
- **Metrics:** IoU, Dice, precision and recall per image, averaged (macro) and with pixels pooled over all images. Boundary F1 uses a `5 px` tolerance and HD95 is the 95th-percentile boundary distance. Instance detection counts a reference egg as found when one predicted component overlaps it with IoU `>= 0.5`. Intervals are 95% bootstrap over images (`5000` resamples).
- **Failure categories** (per image, from IoU `0.8`, precision and recall `0.6`): *good* (IoU `>= 0.8`); *background merged* (precision `< 0.6`, recall `>= 0.6`); *egg missed or partial* (recall `< 0.6`, precision `>= 0.6`); *wrong object* (both `< 0.6`); *mild boundary error* (everything else).

Quantitative results

Metric	Dev, out-of-fold (79)	Eval (20)	All (99)
IoU (macro)	0.776 [0.723, 0.827]	0.809 [0.669, 0.922]	0.783 [0.733, 0.831]
Dice (macro)	0.848 [0.801, 0.889]	0.853 [0.726, 0.958]	0.849 [0.805, 0.889]
Precision	0.869 [0.818, 0.916]	0.846 [0.719, 0.947]	0.864 [0.816, 0.909]
Recall	0.875 [0.827, 0.916]	0.893 [0.777, 0.974]	0.879 [0.835, 0.917]
Boundary F1 (5 px)	0.618 [0.555, 0.682]	0.677 [0.532, 0.803]	0.630 [0.571, 0.687]
HD95, median (px)	35.1	17.9	27.1
IoU, median	0.876	0.926	0.889
IoU, pooled pixels	0.756	0.741	0.753
Images with IoU >= 0.8	61%	80%	65%
Images with IoU < 0.3	6%	15%	8%
Reference eggs detected	76%	91%	79%

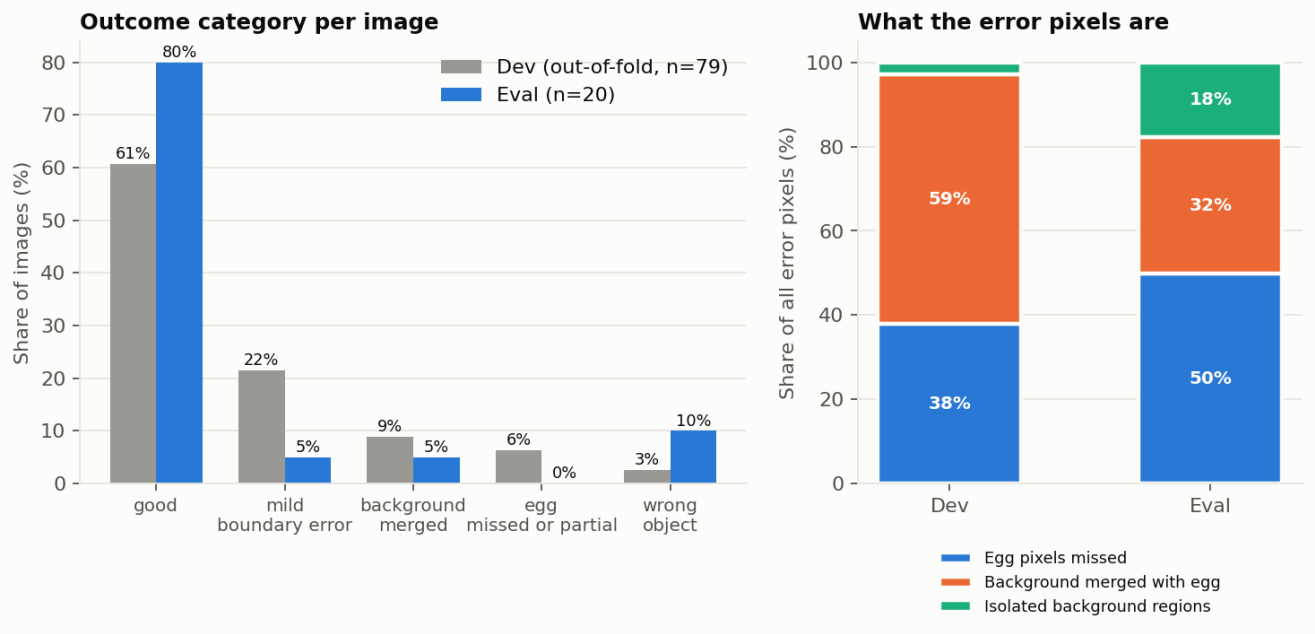
Reading the table.

- **Two groups of images.** Scores are bimodal: median IoU is `0.89` but the mean is `0.78`. The 64 images rated *good* average precision `0.97`, recall `0.97` and median IoU `0.95`; the failures lose most of their score at once (Fig. 1).
- **Dev vs eval.** The eval mean is higher than dev, but the intervals overlap almost completely, so the two cannot be told apart. Treat `0.78` to `0.81` as the expected IoU, not as an improvement on eval.
- **Pooled vs macro.** Pooled IoU is lower than macro IoU because the failures are large regions (a whole carton, a large egg), which weigh more when pixels are summed.



Where the gap comes from



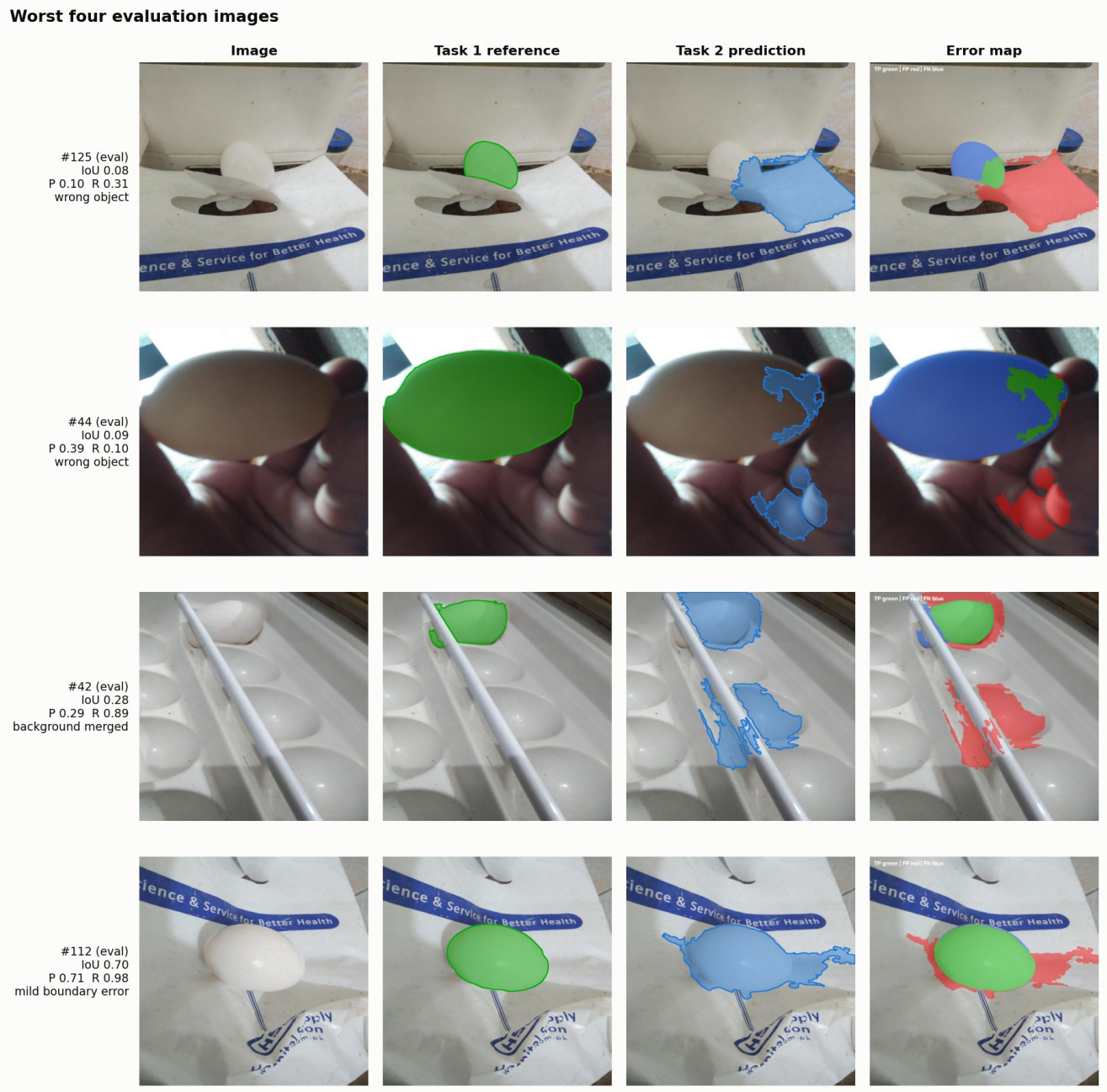


Category	Dev	Eval
good	48	16
mild boundary error	17	1
background merged	7	1
egg missed or partial	5	0
wrong object	2	2

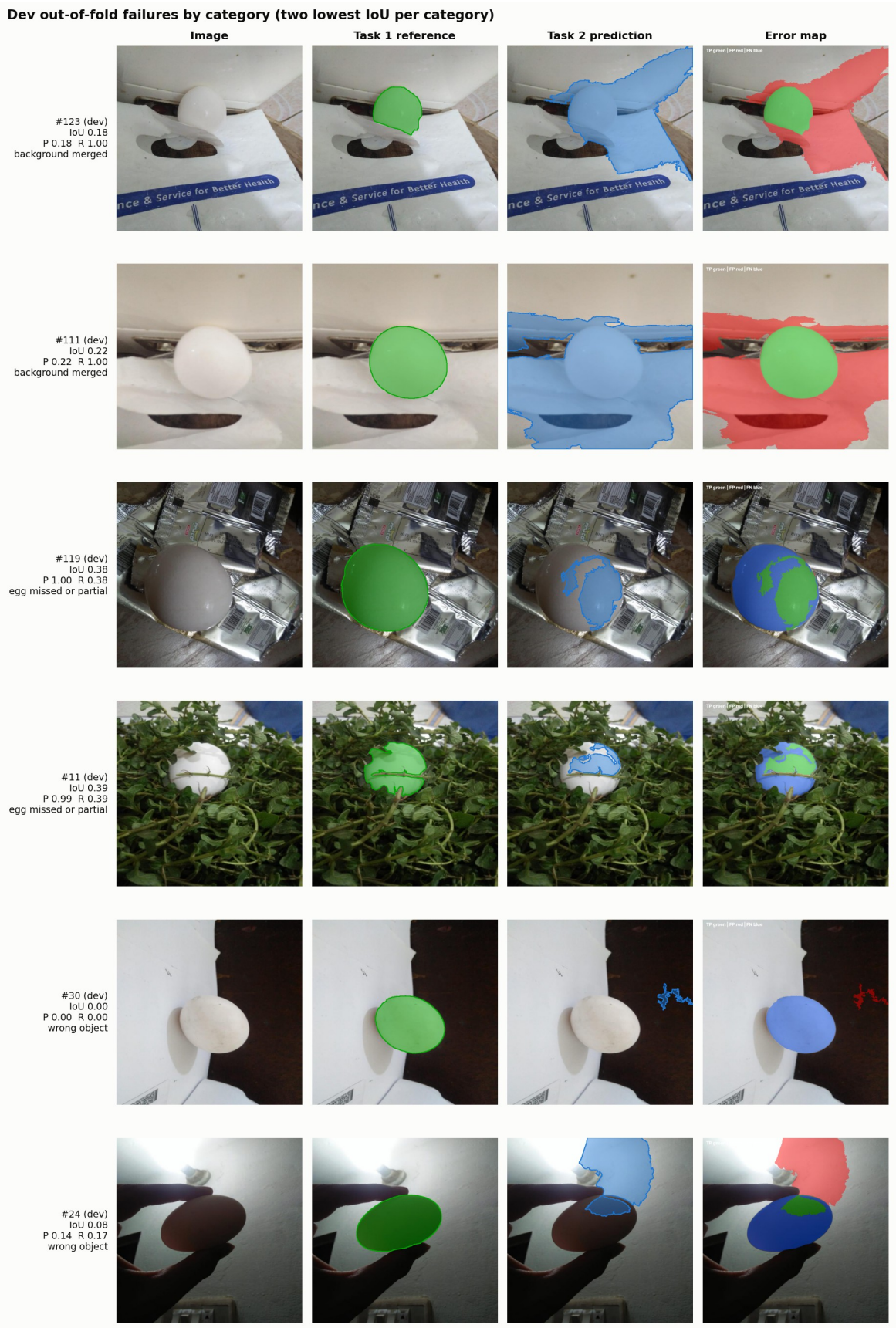
- **Failures are one of two kinds.** Points in Fig. 2 sit either with high recall and low precision (a background region was kept together with the egg) or with low recall (part or all of the egg was lost). Few images sit in between.
- **Isolated background fragments are largely solved.** Only `3%` of dev error pixels (`18%` on eval, from the tray fragments in `#42` and the hand in `#44`) are false positives in components that contain no egg, which is what the shape filter removes.
- **The remaining error is background attached to the egg or egg pixels missing.** In dev, `59%` of error pixels are false positives inside a component that also contains egg, and `38%` are missed egg pixels (eval: `32%` and `50%`). Component-level filtering cannot fix either, because the wrong pixels are inside a component that is otherwise right.
- **Boundaries are only moderately accurate even on good images.** Their median boundary F1 is `0.82` and HD95 about `12 px` on `640 x 640` images. The probability map is predicted at `320` pixels and upsampled, with no edge-aware refinement, so edges are smooth blobs that leak at contacts (Fig. 7, `#158` leaks along the fingers).

Qualitative analysis

Overlay colours in the error maps: green true positive, red false positive (predicted, not egg in Task 1), blue false negative (egg in Task 1, not predicted).

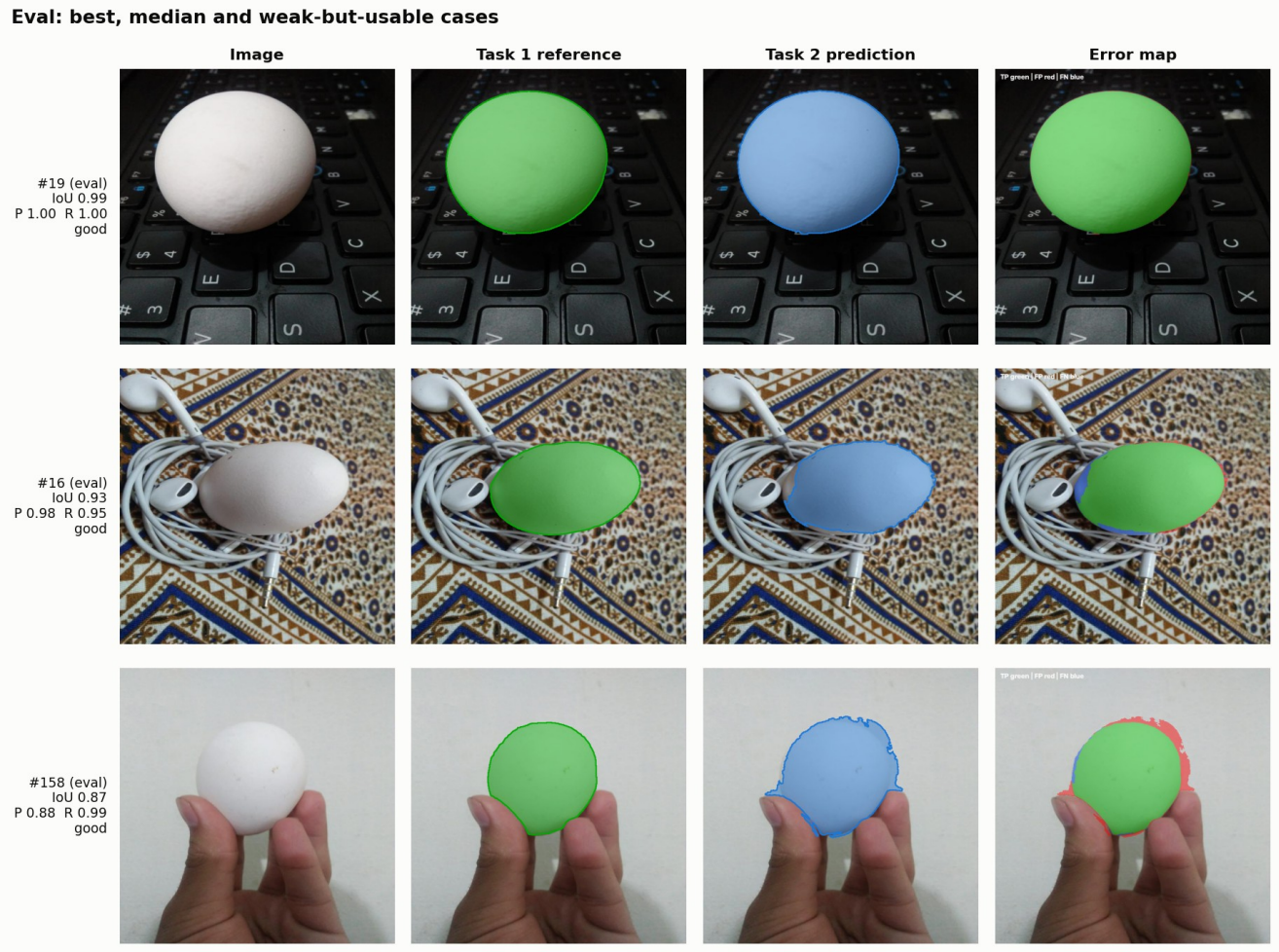


- `#125` **(IoU 0.08, wrong object).** A white egg sits against a white carton. Only a small part of the egg is kept (recall `0.31`) and a carton flap beside it is added (precision `0.10`).
- `#44` **(0.09, wrong object).** An extreme close-up, with the dim egg filling half the frame. The model keeps a small bright patch of the egg (recall `0.10`) and selects the hand (precision `0.39`). Eggs this large are rare: only `5` of `99` images have an egg above `25%` of the frame, and four of them score `0.69` or higher.
- `#42` **(0.28, background merged).** A white egg in a white egg tray with a white rod. The egg is found (recall `0.89`) but tray and rod regions are added (precision `0.29`). It has the lowest egg-to-surround colour difference in the dataset (`4.1` Lab units).
- `#112` **(0.70, mild).** The egg is correct; white plastic bag is attached to its right side (precision `0.71`).

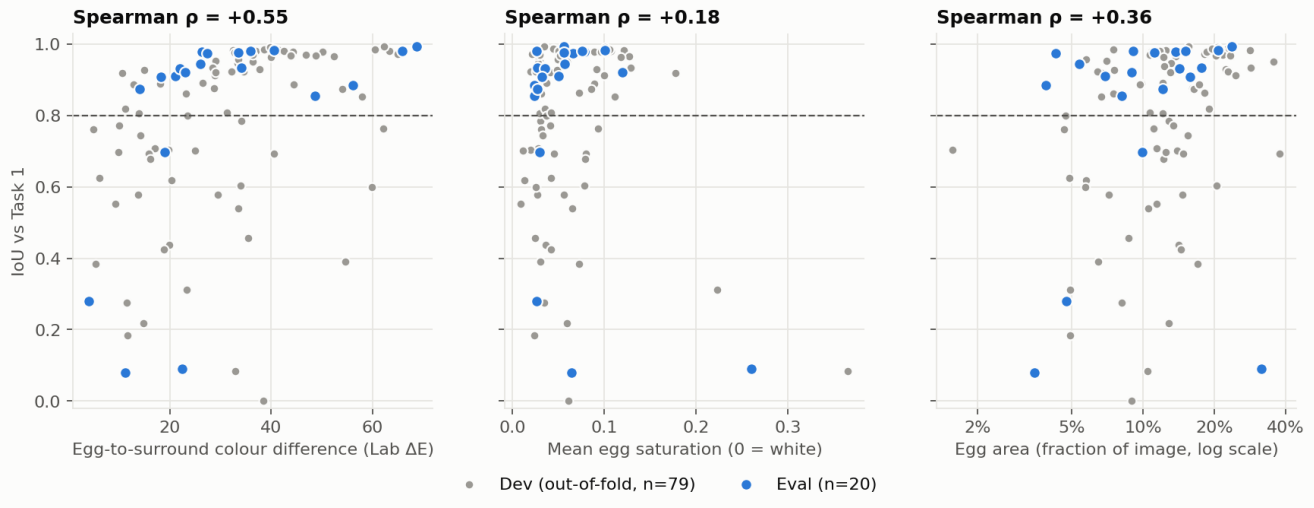


- **Background merged (`#123`, `#111`).** The egg sits on a white carton, and the whole carton is selected (recall `1.00`, precision `0.18` to `0.22`). Both egg and carton are bright, low-texture and similar in colour, so no pixel feature separates them.

- **Egg missed or partial (`#119`, `#11`).** Only a patch of a large egg is kept (precision about `1.0`, recall `0.38` to `0.39`). In `#11` leaves cover part of the egg, and in `#119` the egg has a smooth shading gradient and almost no contrast with the metallic background (ΔE `5.3`).
- **Wrong object (`#30`, `#24`).** In `#30` a white egg on a white wall is not found, and the best-scoring component is a speck elsewhere (the keep-best fallback picks something even when nothing is egg-like). In `#24` a backlit dark egg is overpowered by a glowing wall.



What drives the errors



Egg-to-surround colour difference (Lab ΔE)	Images	Mean IoU	Images with IoU < 0.5
Low (< 22.8)	33	0.64	27%
Middle	33	0.83	9%
High (> 36.4)	33	0.89	6%

- **Contrast with the surroundings is the strongest driver** (Spearman `rho = +0.55`). The `14` images below IoU `0.5` (`11` dev, `3` eval) have median ΔE `19` against `32` for the rest. This is white-on-white, not whiteness itself: egg saturation is only weakly related (`rho = +0.18`, `p = 0.07`).
- **Small eggs fail more often** (`rho = +0.36`). Eggs under `5%` of the image area average IoU `0.56` (`40%` below `0.5`, `10` images), against `0.87` for eggs over `15%`. The close-up `#44` is the exception at the other end.
- **Pixel IoU hides instance errors.** Multi-egg images score well on pixels (mean IoU `0.87` against `0.77` for single-egg), but only `15` of `27` eggs in three-egg images are detected as separate objects. In dev `#142`, `#144` and `#148` three touching eggs form one component with IoU `0.93` to `0.98` and `0` of `3` eggs separated. A union mask cannot count eggs or follow them individually, which Task 4 needs.

Limits of this evaluation

- **The reference is Task 1, not human labels.** The scores measure agreement with OWL-ViT + SAM, and Task 2 is trained on those masks. Task 1 errors (ten false positives were excluded from the split) would carry into every score.
- **The eval set is small.** The `95%` interval on eval IoU is `0.67` to `0.92`.
- **Leakage is reduced, not removed.** The nearest dev image to an eval image scores below the `0.950` CLIP cutoff, but the average is `0.92`, so similar scenes exist on both sides and eval may be somewhat optimistic.
- **The category thresholds** (`0.8`, `0.6`) are a choice for reporting, and the dev failures shaped the method, so dev patterns are not independent evidence.

Potential improvements

Ordered by how much of the observed error each addresses.

1. **Add an egg-shape prior to separate white eggs from white surfaces.** The `14` images below IoU `0.5` are mostly low-contrast and merged cases, where pixel appearance alone cannot work. Candidate ellipses scored by edge support along their perimeter could re-rank or trim the kept component. This has not been tested and is the most direct next experiment.
2. **Split touching eggs into instances.** Needed for counting and for Task 4, and it would also expose merged background regions.
3. **Refine edges at full resolution.** Predict at `640` or refine the mask with an edge-aware step (guided filter or GrabCut seeded from the forest mask) to raise boundary F1 (`0.62` to `0.68`) and cut HD95.
4. **Handle large, partial and very small eggs.** Complete a kept partial region (convex hull or ellipse completion) for cases such as `#119`, `#11` and `#44`, and add training views with large and small eggs.
5. **Drop the keep-best assumption for video.** It always returns a component, so frames without an egg (or the white-egg-on-white-wall failure `#30`) produce a false detection. A confidence threshold on the best component would allow empty output.

Task 4: Video Tracking

The final Task 2 pipeline (Trial 11) is run on the assignment video (https://www.youtube.com/watch?v=Y4JyV94y2_A, saved as `video.mp4`) and extended with candidate filtering and a tracker so that each egg keeps its own identity. Per-frame tracks are in `outputs/task4/tracks.csv`; figures are in `reports/task4/figures/`. The annotated output video is on Google Drive:
https://drive.google.com/file/d/1J1LyZ5amhyR-DFS8jQ2IINm2QukMPneT/view?usp=sharing

The video

60 s, 24 fps, `640 x 360`, 1,440 frames. Eggs appear only from `9.2 s` to `41.6 s` (frames 221-997): two white eggs on a dark glossy table, moved and spun by a hand. Difficulties seen in the footage:

- **Hands cover or hold an egg** for up to about `2.5 s`, and a hand-held egg is lifted off the table.
- **Two near-identical eggs** that can be moved to each other's positions.
- **Egg size:** about `80 x 70` px (`4,400-4,800` px², 2% of the frame), at the small end of the dataset (eggs under 5% of the image averaged IoU `0.56` in Task 3).

Method

Stage 1, per frame (`scripts/task4_detect.py`):

1. **Candidates.** Light, colorless pixels (Lab L `> 165`, HSV saturation `< 70`) are opened with a `5 x 5` kernel; blobs of at least `400` px, mean L `>= 200` and mean saturation `<= 35` survive. Pixels bright in more than 60% of sampled frames (the logo) are masked as a static overlay.
2. **Task 2 segmentation.** Each blob is cropped (side `160-256` px), enlarged to the training scale (`320 x 320`) and passed through the final Task 2 pipeline (Auto-Context Random Forest, hole filling, component features). The component overlapping the blob most is kept and mapped back. If it grows beyond `1.35x` the blob (it leaked into table sheen) it is trimmed to the blob's neighbourhood, and as a last resort the blob itself is used. A `9 x 9` opening removes spikes from motion blur.
3. **Validation.** A region is *strong* if its area is `1,500-9,000` px², aspect `<= 1.9`, solidity `>= 0.9`, mean L `> 200` and saturation `< 40`. Compact, white regions of at least `300` px² (`600` px² when used by the tracker) are *weak* (partly hidden eggs). Everything else is rejected.

Stage 2, tracking (`scripts/task4_track.py`):

- **Association.** A constant-velocity Kalman filter per egg; detections are matched by Hungarian assignment on distance (gate `60 px` plus `12 px` per missed frame, capped at `160`) and area ratio. Predicted velocity decays by `0.85` per hidden frame.
- **Occlusion.** A track survives `72` frames (`3 s`) without a detection and is drawn as a hidden egg at its predicted position. It stays available for re-identification for `360` frames (`15 s`).
- **Re-identification.** A strong detection that matches no live track first reclaims the nearest lost egg of plausible size, anywhere in the frame; a new egg is created only if its area is at least `70%` of the typical egg area. A track must collect `5` detections before it is reported, and a scene cut (mean frame change above `12`) ends all tracks.
- **Numbering.** Ids are assigned left to right at first sighting. Hidden or partial eggs are labeled in the video.

Outputs: annotated video (colored masks, ids, state, short trails), `tracks.csv` (frame, id, state, centroid, area, source), and the figures below. Detection takes about `20 min` on CPU (plus `2 min` to train and cache the Task 2 model once); tracking and rendering take about a minute.

How the approach developed

Step	Finding	Change
Task 2 pipeline on 8 frames, as is	Eggs outlined well, but `2-6` components kept per frame against 2 eggs (sheen, reflections, shirt checks); eggs under the hand came out as fragments	Add candidate proposals and validation in front of the segmenter; plan a tracker for occlusion

Step	Finding	Change
First size limits (from a visual estimate of 40-50 px eggs)	Wrong: measured blobs gave 80 x 70 px, mean L about 225-242 against 175-212 for shirt and sheen	Re-measured; limits, crop size and whiteness thresholds set from the blob table
Frame 561	Task 2 component leaked into table sheen (5,863 px against a clean 4,147 px blob) and was rejected, losing the egg	Trim the component to the blob's neighbourhood, else use the blob (70 records use task2+blob, 12 use blob)
First tracker	4 ids instead of 2: the held egg's track matched 300 px shirt fragments and its predicted position drifted; when the egg reappeared 400 px away the 320 px re-id gate failed	Ignore weak detections under 600 px, faster velocity decay, 15 s re-id horizon without a distance limit, birth only for near-full-size eggs
Rendering	Red and green overlays are a color-blind trap; trails drew across re-id jumps and as scribbles inside resting eggs	Blue and orange from a validated palette; trails break at jumps, are cleared while hidden and drawn only when the egg has moved 30 px

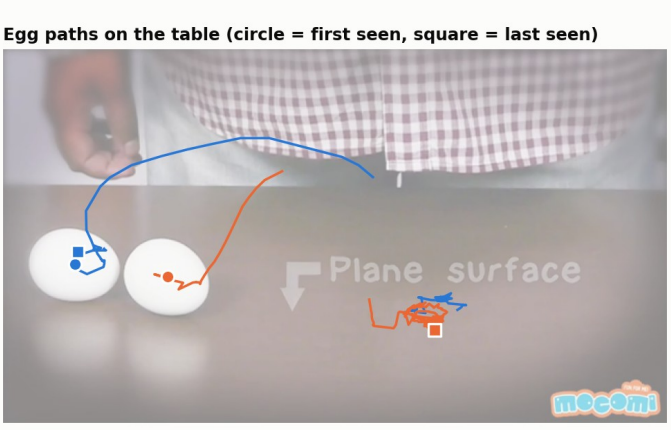
Results

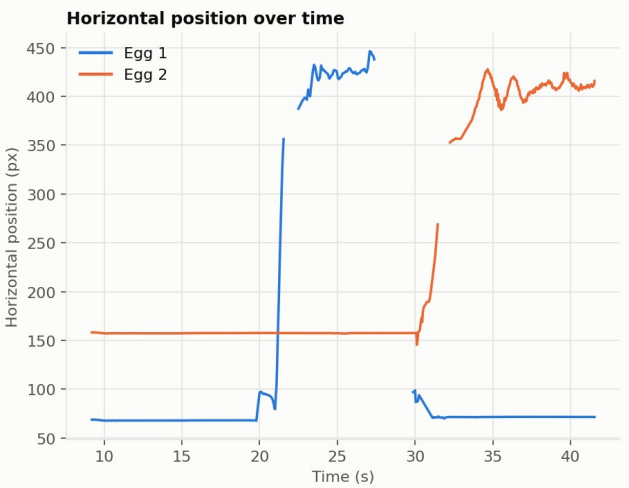
Tracking. Two ids are reported, both spanning the whole egg segment (frames 221-997, 32.4 s), with no gaps.

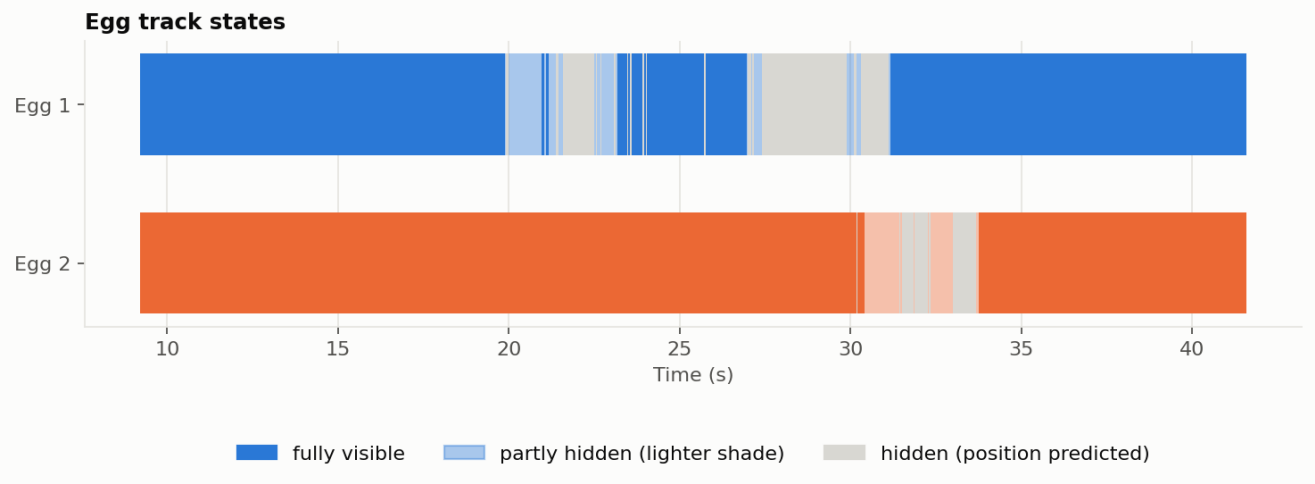
	Egg 1	Egg 2
Fully visible	596 frames (77%)	696 frames (90%)
Partly hidden	63	45
Hidden (position predicted)	118	36
Longest hidden stretch	59 frames (2.5 s)	16 frames (0.7 s)
Largest jump between detections	346 px (an egg carried under the hand)	104 px

Both eggs are fully visible together in 534 of 777 egg frames (69%). Of 1,400 detections, 1,318 come from the Task 2 pipeline directly.

Identity. The tracks show the hand events in the data (Fig. 1): Egg 2 stays at about x = 157 px until 30 s and is then carried to the right, while Egg 1 stays at about x = 68 px until 20 s, is carried to the right (x = 400-450), is hidden while the hand holds it (up to 2.5 s at a time, Fig. 2) and is back at about x = 71 px from 31 s. Whether each id stays on the same physical egg through these hand events was checked by watching the output video; that check is manual, is not reproduced by the scripts, and no figure of it is included. There is no annotated ground truth, so id switches and MOTA are not measured.

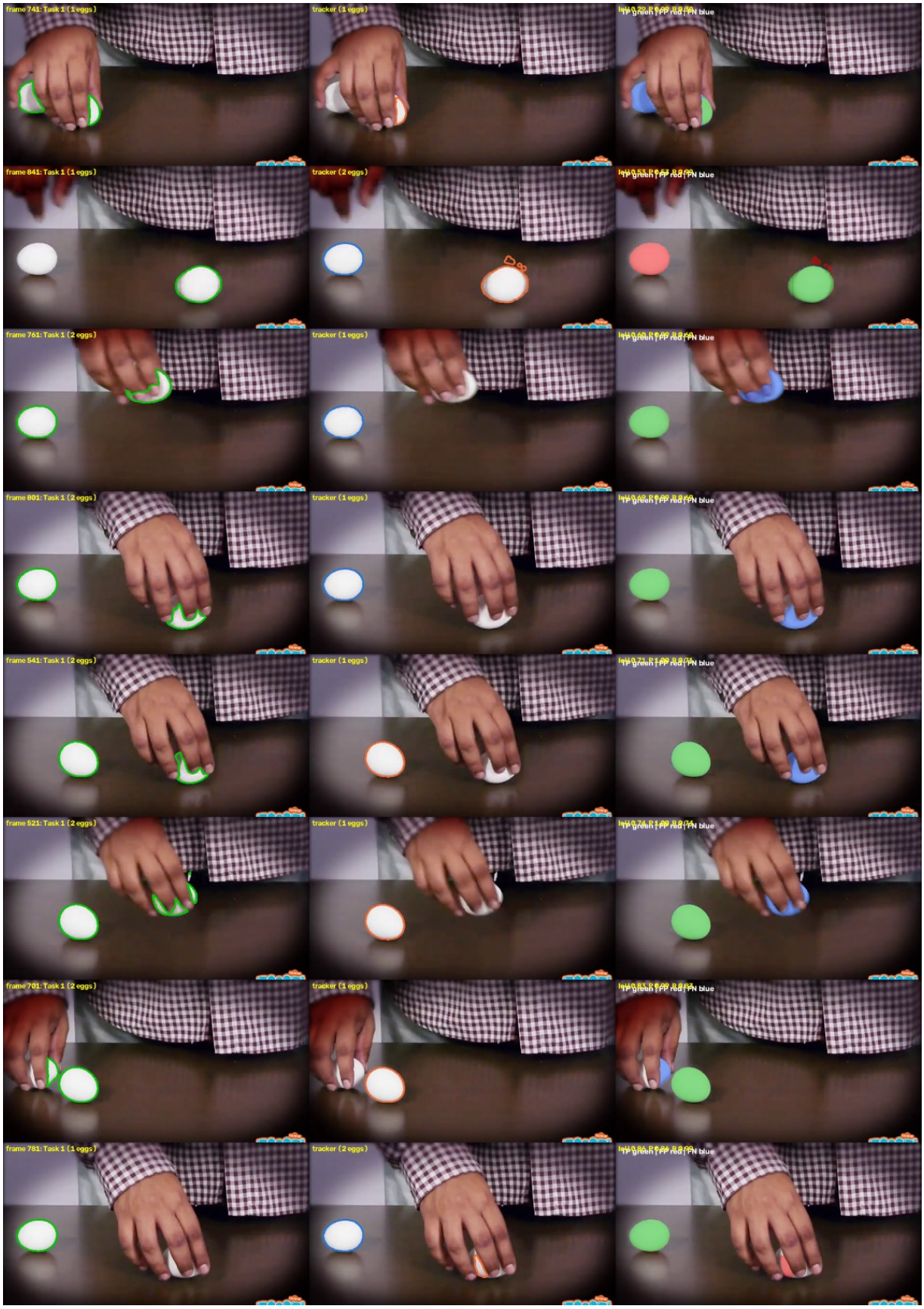






Segmentation against Task 1. Task 1 (OWL-ViT + SAM) was run on every 20th egg frame (39 frames, 221-981) and compared with the union of tracked masks, as in Task 3.

Measure	Value
Macro IoU / Dice	0.913 / 0.946
Macro precision / recall	0.975 / 0.938
Median frame IoU	0.984 (29 of 39 frames at >= 0.95, 8 below 0.9, 1 below 0.5)
Frames with both eggs found by both methods	IoU 0.979 (29 frames)
Fully visible eggs: best IoU with a Task 1 egg	mean 0.967, median 0.987 (n = 65; 98% at >= 0.5)
Partly hidden eggs	mean 0.586 (n = 5; 60% at >= 0.5)
Tracked eggs matched to a Task 1 egg at IoU >= 0.5	67 of 70 (Task 1 found 73)



The `8` frames below `0.9` (Fig. 3) are almost all hand occlusions, and neither method is right there: the tracker reports a hand-held egg as hidden or as a small crescent (tracker egg count below Task 1's in frames `521, 541, 701, 761, 801`), while Task 1 outlines it with ragged edges through the finger gaps. In frame `841` Task 1 misses the left egg altogether, so the tracker's second egg is counted against it (precision `0.53`); in frame `781` the tracker's small crescent

at the fingers has no counterpart in Task 1.

Limitations

- **Hidden eggs have no mask.** For 154 egg-frames the position is a prediction, not a measurement, and an egg held in the hand is not segmented.
- **Settings were set on this one video.** Brightness, size and aspect limits come from this footage and may need changing for other scenes; the Task 2 model was not retrained on video frames.
- **Re-identification assumes no new egg enters.** A lost egg is reclaimed by the next strong detection within 15 s, so a genuinely new egg appearing after one is lost would take its id. Eggs touching each other are not split; they were never merged in this video.
- **Spinning eggs are over-segmented.** The blur halo makes a spinning egg's mask 5,300–6,800 px² against 4,700 at rest.
- **Reference is imperfect.** Task 1 is not ground truth here (it misses an egg in frame 841 and misses or fragments hand-held eggs), so the 0.913 is agreement, not accuracy. The 39 frames are a regular sample, not the worst cases.
- **Speed.** About 1.5 s per egg frame on CPU, so this is an offline method.

Potential improvements

- **Complete hidden eggs.** Fit an ellipse of the typical egg size at the predicted position so hand-held eggs get an amodal mask, and split touching eggs by a size prior.
- **Handle blur.** Use an elongated-ellipse model or temporal smoothing of the mask for spinning eggs.
- **Measure tracking properly.** Annotate egg centers on a subset of frames and report id switches, fragmentation and MOTA, instead of relying on watching the video.
- **Speed up.** Reuse the previous frame's mask as a prior and segment only changed regions.

Reproduce

```
python scripts/task4_detect.py      # candidates and Task 2 segmentation for every frame (about 20 min,
trains the model on first use)
python scripts/task4_track.py       # tracking, outputs/task4/tracks.csv and tracked_eggs.mp4
python scripts/task4_plots.py       # reports/task4/figures
python scripts/task4_evaluate.py    # Task 1 comparison on sampled frames (needs the Task 1 models; about 20
min on CPU)
```

Setup and Reproduction

All code, logs, tables and figures are in the repository: https://github.com/AnantSingla412/Nosh-CV. The README has the full step-by-step commands for every task.

Setup (Windows PowerShell)

Python 3.10 or newer is recommended.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Where each task lives

Task	Scripts	Main outputs
1. Pretrained segmentation	`task1_segment.py`	`outputs/task1_final/` (masks and overlays)
2. Classical segmentation	`task2_split.py`, `task2_similarity_analysis.py`, `task2_classical.py`, `task2_grabcut.py`, `task2_pixel_classifier*.py`, `task2_component_filter.py`, `task2_final_eval.py`	`Dataset/task_2/` (split), `reports/task2_experiment_log.md`, `outputs/task2/final_eval/`
3. Evaluation and analysis	`task3_evaluation.py`	`reports/task3/` (tables and figures), `reports/task3_evaluation_report.md`
4. Video tracking	`task4_detect.py`, `task4_track.py`, `task4_plots.py`, `task4_evaluate.py`	`outputs/task4/` (video, tracks, tables), `reports/task4/figures/`

Run order

The assignment video is saved as `video.mp4` in the repository root (it is not committed).

```
python scripts/task1_segment.py
python scripts/task2_split.py --clip-threshold 0.950
python scripts/task2_pixel_classifier_autocontext.py
python scripts/task2_component_filter.py
python scripts/task2_final_eval.py
python scripts/task3_evaluation.py
python scripts/task4_detect.py
python scripts/task4_track.py
python scripts/task4_plots.py
python scripts/task4_evaluate.py
```

The earlier Task 2 trials (Otsu, adaptive threshold, GrabCut and the Random Forest variants) each have their own script and run command listed in the README. Random seeds are fixed, so results should repeat closely; the slow steps are the Task 2 Auto-Context run, the Task 4 detection pass (about 20 minutes on CPU) and the Task 1 comparison in Task 4 (about 20 minutes on CPU).